Smart Courier Interview Guide

1. Project Summary

This project is a microservices-based courier management system built with Spring Boot.

It is designed around separate services for:

- authentication
- delivery management
- tracking
- admin/dashboard features
- API routing
- centralized configuration
- service discovery

The project now runs in Docker as a complete local containerized environment.

2. What Problem This Project Solves

The system models a courier/logistics platform where:

- customers can sign up and log in
- customers can create and view deliveries
- admins can manage delivery status
- delivery progress can be tracked with tracking events
- documents and proof of delivery can be uploaded
- admins can view dashboard and hub information

Instead of building all of this in one monolithic application, the project separates concerns into multiple services.

3. Services In This Project

API Gateway

Path: api-gateway/src/main/java/com/smartcourier/gateway/controller/GatewayController.java

Purpose:

- single entry point for client requests
- forwards requests to internal services
- validates JWT token
- extracts user email and role from token
- passes user context through headers like `X-User-Email` and `X-User-Role`
- aggregates Swagger/OpenAPI docs from internal services

Important interview point:

The client should talk to the gateway, not directly to internal services. This is a common microservice pattern because it centralizes authentication, routing, and request handling.

Auth Service

Path: auth-service/src/main/java/com/smartcourier/auth/controller/AuthController.java

Purpose:

- user signup
- user login
- JWT generation

Main endpoints:

- `POST /auth/signup`
- `POST /auth/login`

Important interview point:

Auth service is separated so security logic is isolated from business services.

Delivery Service

Path: delivery-service/src/main/java/com/smartcourier/delivery/controller/DeliveryController.java

Purpose:

- create deliveries
- get delivery by ID
- get delivery by tracking number
- get current user deliveries
- cancel delivery
- admin status updates

Main endpoints:

- `POST /deliveries`
- `GET /deliveries/my`
- `GET /deliveries/{id}`
- `GET /deliveries/track/{trackingNumber}`
- `PUT /deliveries/{id}/status`
- `PUT /deliveries/{id}/cancel`
- `GET /deliveries/all`

Important interview point:

This service is the core operational service for shipment lifecycle.

Tracking Service

Path: tracking-service/src/main/java/com/smartcourier/tracking/controller/TrackingController.java

Purpose:

- add tracking events
- fetch tracking history
- upload documents
- store delivery proof

Main endpoints:

- `POST /tracking/events`
- `GET /tracking/{trackingNumber}`
- `POST /tracking/documents/upload`
- `GET /tracking/documents/{trackingNumber}`
- `POST /tracking/proof`
- `GET /tracking/proof/{trackingNumber}`

Important interview point:

This service handles file and proof-related workflows separately from delivery creation logic.

Admin Service

Path: admin-service/src/main/java/com/smartcourier/admin/controller/AdminController.java

Purpose:

- dashboard statistics
- delivery resolution
- reports
- hub management

Main endpoints:

- `GET /admin/dashboard`
- `GET /admin/deliveries`
- `PUT /admin/deliveries/{id}/resolve`
- `GET /admin/reports`
- `POST /admin/hubs`
- `GET /admin/hubs`
- `GET /admin/hubs/active`
- `PUT /admin/hubs/{id}/deactivate`

Important interview point:

Admin functionality is separated to keep internal operational concerns isolated from customer-facing workflows.

Eureka Server

Path: eureka-server/src/main/java/com/smartcourier/eureka/EurekaServerApplication.java

Purpose:

- service discovery
- keeps track of which services are running and where they are located

Important interview point:

In microservices, service discovery avoids hardcoding service locations everywhere.

Config Server

Path: config-server/src/main/java/com/smartcourier/configserver/ConfigServerApplication.java

Purpose:

- centralized configuration management
- serves service config files from the shared `config-files` folder

Important interview point:

This prevents configuration duplication across all services.

4. Supporting Infrastructure

MySQL

Purpose:

- stores service data

Databases used:

- auth_db
- delivery_db
- tracking_db
- admin_db

RabbitMQ

Purpose:

- async messaging between services
- useful for event-based communication like delivery updates

Zipkin

Purpose:

- distributed tracing
- helps trace requests across multiple services

5. How Request Flow Works

Example flow for customer login and delivery creation:

1. Client sends login request to API Gateway.
2. Gateway forwards request to Auth Service.
3. Auth Service returns JWT token.
4. Client calls delivery APIs through Gateway with `Authorization: Bearer <token>`.
5. Gateway validates the token.
6. Gateway extracts user identity and role.
7. Gateway forwards the request to Delivery Service with internal headers.
8. Delivery Service performs business logic and returns response.

Example flow for admin action:

1. Admin logs in and gets JWT.
2. Admin calls Gateway endpoint.
3. Gateway validates token and forwards role.
4. Admin Service or Delivery Service checks `X-User-Role=ADMIN`.
5. Protected operation is executed.

6. Why Port 8888 Shows White Label Error Page

Port `8888` is the Config Server.

That server is meant to serve configuration data to other services, not act like a user-facing website.

So when you open:

- `http://127.0.0.1:8888`

you may see a Spring Boot white-label page or a generic error page.

That does **not** mean the Config Server is broken.

It usually means:

- the server is running
- but there is no user-facing controller mapped to `/`

That is normal.

Better checks for Config Server are:

- `http://127.0.0.1:8888/auth-service/default`
- `http://127.0.0.1:8888/delivery-service/default`

Those should return configuration JSON if the Config Server is working correctly.

7. Why Root `/` On Gateway May Show Nothing

The gateway is designed mainly for proxied APIs and Swagger UI.

If you open:

- `http://127.0.0.1:8080/`

there may be no useful root endpoint.

The better test URL is:

- `http://127.0.0.1:8080/swagger-ui/index.html`

That page confirms the gateway is serving the aggregated API docs UI.

8. What We Changed To Dockerize The Project

This is the most important part for interview explanation.

Before Dockerization

Originally, the project had common local-machine assumptions:

- services expected `localhost` for MySQL
- services expected `localhost` for RabbitMQ
- services expected `localhost` for Eureka
- services expected `localhost` for Zipkin
- config server originally depended on external Git-based config behavior
- Docker builds depended on Maven wrapper bootstrapping

Those assumptions work on a single machine, but fail inside Docker because:

- inside a container, `localhost` means that same container only
- one container cannot reach another using `localhost`

Dockerization Changes We Made

1. Reworked the Config Server to use local native config

File:

- config-server/src/main/resources/application.yml

What we changed:

- switched Config Server to `native` profile
- mounted `./config-files` into the container
- used that folder directly instead of remote Git clone behavior

Why:

- more stable for local Docker use
- avoids startup failures caused by Git/network dependency

2. Parameterized service configuration with environment variables

Files:

- config-files/auth-service.yml

- config-files/delivery-service.yml
- config-files/tracking-service.yml
- config-files/admin-service.yml
- config-files/api-gateway.yml

What we changed:

- replaced hardcoded `localhost` with placeholders like:
  - `${MYSQL_HOST:localhost}`
  - `${RABBITMQ_HOST:localhost}`
  - `${EUREKA_SERVER_URL:http://localhost:8761/eureka/}`
  - `${ZIPKIN_ENDPOINT:http://localhost:9411/api/v2/spans}`

Why:

- local development still works with default localhost values
- Docker containers can override those values with service names

This is a strong interview point:

We kept the same codebase usable in both environments by externalizing environment-specific config.

3. Updated application-local fallback configs too

Files:

- auth-service/src/main/resources/application.yml
- delivery-service/src/main/resources/application.yml
- tracking-service/src/main/resources/application.yml
- admin-service/src/main/resources/application.yml
- api-gateway/src/main/resources/application.yml
- eureka-server/src/main/resources/application.yml

What we changed:

- made the embedded configs environment-variable aware as fallback

Why:

- if Config Server is temporarily unavailable during startup, services still have valid fallback config

4. Updated Docker Compose networking and dependencies

File:

- docker-compose.yml

What we changed:

- defined all infrastructure and service containers
- passed correct environment variables for Docker networking
- mounted config files into Config Server

- persisted MySQL data in a volume
- persisted tracking uploads in a volume
- removed the hard dependency on RabbitMQ health status
- removed host binding for MySQL port `3306` because it conflicted with host MySQL

Why:

- Docker containers should communicate using service names like:
  - `mysql`
  - `rabbitmq`
  - `eureka-server`
  - `config-server`

5. Switched Docker builds from Maven Wrapper to Maven image

Files:

- auth-service/Dockerfile
- delivery-service/Dockerfile
- tracking-service/Dockerfile
- admin-service/Dockerfile
- api-gateway/Dockerfile
- config-server/Dockerfile
- eureka-server/Dockerfile

What we changed:

- changed build stage to use `maven:3.9.9-eclipse-temurin-21`
- used `mvn -B -DskipTests package`

Why:

- Maven wrapper download was unstable during Docker build
- official Maven image makes the build more reliable

6. Added `.dockerignore` files

Why:

- avoids sending `target/`, logs, and IDE metadata into Docker build context
- makes builds cleaner and faster

7. Fixed a real Java compile issue

File:

- admin-service/src/main/java/com/smartcourier/admin/dto/DashboardStats.java

What we changed:

- removed duplicate constructor/getter/setter code already covered by Lombok

Why:

- full clean Docker build exposed a compilation error that may not have been obvious in local IDE runs

This is also a strong interview point:

Containerization often reveals hidden environment or build problems because it forces a clean, reproducible build.

9. Why Dockerization Was Necessary

If someone asks why we dockerized it, explain:

- easy onboarding for other developers
- same runtime environment across machines
- avoids “works on my machine” issues
- reproducible builds
- easier deployment path later
- infrastructure dependencies like MySQL and RabbitMQ come up together

10. What “Working In Docker” Means Here

For this project, “working in Docker” means:

- all services build successfully as images
- all containers start through one compose command
- service-to-service communication works by container DNS names
- external infrastructure is available through containers
- ports are exposed for browser/API access

11. Exact Run Flow

Use:

```
cd D:\SPRINT1\docker_smartcourier
docker compose up -d
docker compose ps
```

Then wait 1-2 minutes because Java services need warm-up time.

Useful URLs:

- Gateway Swagger UI: `http://127.0.0.1:8080/swagger-ui/index.html`
- Eureka: `http://127.0.0.1:8761`
- Config Server: `http://127.0.0.1:8888`
- RabbitMQ UI: `http://127.0.0.1:15672`
- Zipkin: `http://127.0.0.1:9411`

12. Common Confusions And Their Meaning

White-label error on 8888

Meaning:

- Config Server is running
- there is no UI endpoint on `/`
- not necessarily a problem

Nothing visible on 8080 root

Meaning:

- gateway root may not expose content
- use Swagger UI path instead

Service shows `Up` but page not reachable immediately

Meaning:

- container is started
- application may still be booting
- Java microservices can take 30-90 seconds

Docker worked once and then Docker API returned 500/502

Meaning:

- Docker Desktop itself was unstable
- not necessarily a project issue

13. Best Interview Explanation You Can Say

You can say this:

"This project is a Spring Boot microservices-based Smart Courier platform. It has separate services for authentication, delivery, tracking, admin operations, centralized configuration, service discovery, and an API gateway. I containerized the entire system using Docker Compose so that all services and dependencies like MySQL, RabbitMQ, and Zipkin can run together in a reproducible environment."

"The main challenge in Dockerizing it was that the project originally used localhost-based assumptions. Inside Docker, localhost points to the same container, so I changed the configuration to use environment-driven service discovery. I also switched the Config Server to use a local mounted config repository, updated Dockerfiles to use a stable Maven image for builds, added persistent volumes, and fixed a compile issue that was exposed by the clean container build."

"After these changes, the whole stack builds and starts under Docker, and the application is reachable through the API Gateway and supporting service ports."

14. Good Technical Keywords To Mention

- microservices architecture

- API Gateway pattern
- centralized configuration
- service discovery
- environment-based configuration
- container networking
- Docker Compose orchestration
- persistent volumes
- clean reproducible builds
- asynchronous messaging with RabbitMQ
- distributed tracing with Zipkin

15. If They Ask “What Did You Personally Do?”

You can answer:

- analyzed the startup dependencies between services
- removed hardcoded localhost assumptions
- made configuration environment-aware
- connected services using Docker Compose networking
- replaced unstable wrapper-based Docker builds with Maven-image builds
- added persistent volumes for data and file uploads
- fixed compile issues discovered by clean Docker builds
- validated the system by bringing up the full stack and testing exposed endpoints

16. Final Status

Current confirmed working checks:

- Eureka is reachable on port `8761`
- API Gateway Swagger UI is reachable on port `8080`
- RabbitMQ UI is reachable on port `15672`
- Docker Compose shows services as running

17. Recommended Next Demo For Interview

If you want to demonstrate the project live:

1. show `docker compose ps`
2. open Eureka
3. open Swagger UI through the gateway
4. explain that all traffic goes through the gateway
5. demonstrate signup and login
6. use JWT token to call a delivery endpoint

18. About PDF

This guide is saved as Markdown so you can open it in VS Code, GitHub-style preview, or convert it to PDF easily using editor export or print-to-PDF.

File:

- INTERVIEW_GUIDE.md